Information Retrieval

WS 2022 / **2023**

Lecture 12, Tuesday January 31st, 2023
(Knowledge Graphs, SPARQL, Translation to SQL)

Prof. Dr. Hannah Bast
Chair of Algorithms and Data Structures
Department of Computer Science
University of Freiburg

UNI FREIBURG

Overview of this lecture

- Organizational
  - Your experiences with ES11 — Logistic Regression
- Content
  - Knowledge graphs + SPARQL — explanation + examples
  - Databases + SQL — explanation + examples
  - SQLite — a lightweight database
  - SPARQL to SQL — algorithm + example
  - Performance — join operations and join order

ES12: Implement the SPARQL → SQL translation and use it to process SPARQL queries with Python+SQLite

Experiences with ES11 1/2

- Summary / excerpts

Sorry!

  - Interesting lecture, 2h 30m was too long, many skipped ES12

    "Sorry I could not make time for this exercise sheet"

    "I liked that 'out-of-focus' parts were already implemented"

    "I finally got Logistic Regression … fifth time a charm"

    "I feel like my linear algebra is not rusty anymore!"

    "Numpy, especially multiplication (* vs dot) still annoy me"

    "Epochs were defined on ES11, but not on the slides"

    "The code template did not meet the flake8 requirements"

    "Even the extended lecture time is overdrawn regularly"

    "A close friend of MrBeast said he was very anti-social"

Experiences with ES11 2/2

- Is our brain inherently a (digitizable) neural network?
  - A variety of opinions on a fascinating topic

    "Yes, but I don't think it uses gradient descent"

    "It might be a neural network but not necessarily digitizable"

    "I don't think there is more than what is physically measureable"

    "Too many things are saved in multiple weird ways"

    "The brain is a lot more complex than a neural network"

    "I believe consciousness is not just a neural network …
    but maybe that's just my brain trying to be special"

    My two cents: when you look back in history, for any topic X, before it was clear how it worked technically, humans always filled the gap in understanding with mystical stories

Knowledge Graphs and SPARQL 1/7

- What is a knowledge graph
  - A knowledge graph (or: knowledge base) is a database of statements about entities and their relations
  - Critical: **unique** identifiers for each entity and predicate
  - A common format / schema is to express all statements as subject predicate object triples:

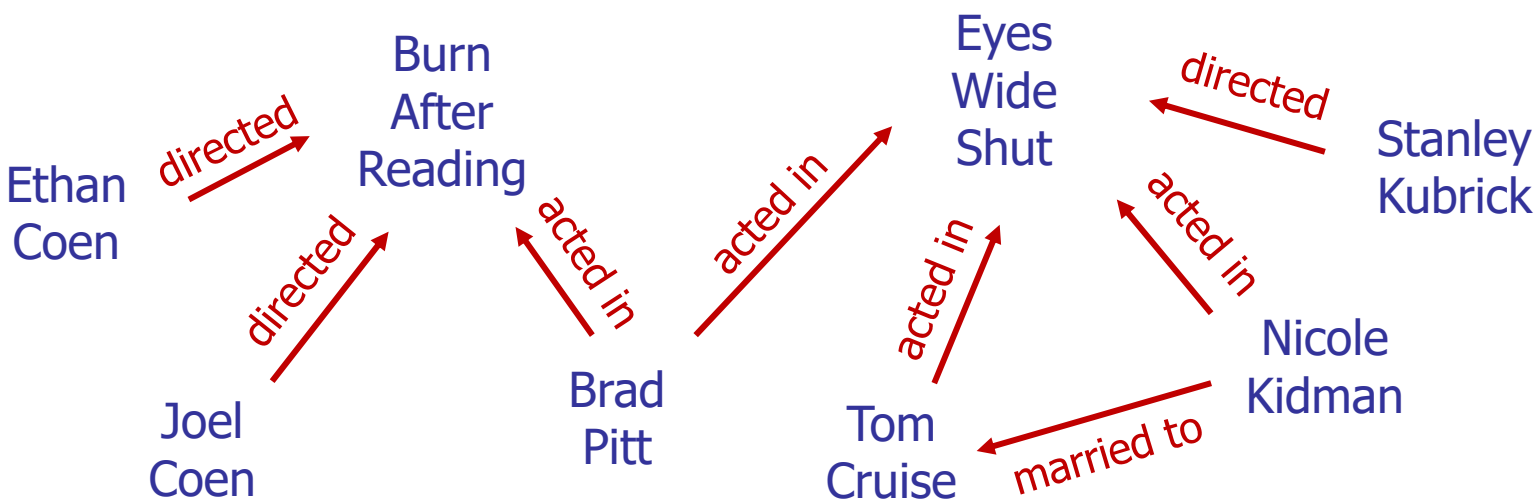
Nicole_Kidman	acted_in	Eyes_Wide_Shut
Brad_Pitt	acted_in	Burn_After_Reading
Joel_Coen	directed	Burn_After_Reading
Ethan_Coen	directed	Burn_After_Reading
Tom_Cruise	acted_in	Eyes_Wide_Shut
Brad_Pitt	acted_in	Eyes_Wide_Shut
Nicole_Kidman	married_to	Tom_Cruise
Stanley Kubrick	directed	Eyes_Wide_Shut

Knowledge Graphs and SPARQL 2/7

- Graph representation
  - The triples of a KG form a **directed graph** as follows:

    Nodes: Each distinct entity is one node

    Edges: Each triple is a directed edge from the subject node to the object node



Knowledge Graphs and SPARQL 3/7

- Wikidata & Freebase
  - Freebase was the largest general-purpose KG until 2018

    Started by Metaweb in 2007, acquired by Google in 2010 for the tiny amount of $ 99,999,999

    Freebase has become read-only in March 2015 and Wikidata has taken over as **the** standard general-purpose KG

    Freebase final size: **3 billion** triples on **60 million** entities
  - Wikidata is like Wikipedia for knowledge graphs: anybody can contribute (with some amount of editorial control)

    Current size: **18 billion** triples on **87 million** entities

    The dataset for ES12 is an extract from Wikidata, much simplified for your convenience

Knowledge Graphs and SPARQL 4/7

- Reification
  - Restriction to triples is no real restriction: n-ary relationships can also be represented as triples:

Nicole Kidman	married_to	xyz
xyz	married_to	Tom Cruise
xyz	start_time	24 Dec 1990
xyz	end_time	2001
xyz	place_of_marriage	Telluride

xyz is a so-called **mediator node**, the purpose of which is to serve as a link between the entities it connects.

For simplicity, the dataset for ES12 has no such nodes

Knowledge Graphs and SPARQL 5/7

- What is SPARQL
  - The standard query language for knowledge bases

    SPARQL = **S**PARQL **P**rotocol **A**nd **R**DF **Q**uery **L**anguage
  - Example query in natural language:

    Married couples who acted together in at least one movie
  - The same query expressed in (slightly simplified) SPARQL

```
SELECT ?person1 ?person2 ?film WHERE {
  ?person1   acted_in   ?film  .
  ?person2   acted_in   ?film  .
  ?person1   married_to  ?person2  .
}
```

Knowledge Graphs and SPARQL 6/7

- SPARQL syntax
  - In the lecture today, we use a simplified syntax

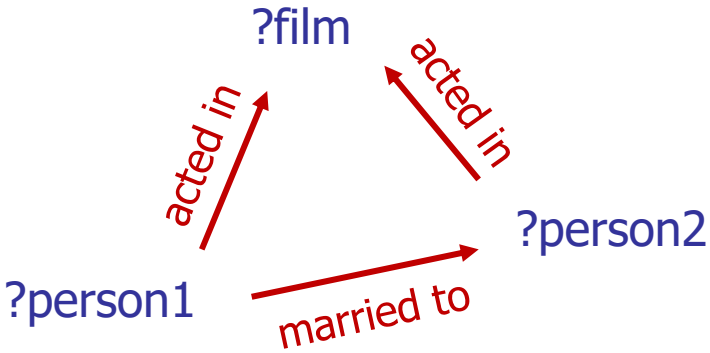
    In "real" SPARQL, the names of subjects / predicates / objects may contain whitespace and are surrounded by <…>
  - The actual SPARQL syntax is slightly more complicated and has many more features

    In particular, it involves namespace prefixes, so that names can be made globally unambiguous

    See the Wikipedia page or the W3C specification if you are interested

Knowledge Graphs and SPARQL 7/7

- SPARQL queries as graph patterns
  - A SPARQL query can be viewed as a sub-graph with variables at some nodes or edges
  - Solving the query then amounts to finding all matches of that (small) sub-graph in the (large) knowledge graph



Each distinct match corresponds to one assignment of values to the variables such that the corresponding sub-graph exists in the knowledge graph

Databases and SQL 1/4

- Introduction
  - The triples from a knowledge graph can also be stored in an ordinary database

    This is what we do in the lecture today and for ES12
  - The standard query language for databases is SQL

    SQL = **S**tructured **Q**uery **L**anguage
  - On the following slides, let us recap the basics from databases and SQL via a few examples

Databases and SQL 2/4

- What is a database
  - For this lecture, a database is a collection of tables, where each table has a fixed number of columns
  - For example, we could have one table for each predicate from our knowledge graph, with two columns each

Table for "acted_in" predicate

person	film
Brad Pitt	Burn After Reading
Tom Cruise	Eyes Wide Shut
Nicole Kidman	Eyes Wide Shut
…	…

Table for "married_to" predicate

person1	person2
Nicole Kidman	Tom Cruise
Ellen DeGeneres	Portia de Rossi
Pythagoras	Nero
…	…

For ES12, you should work with **one** big table for the whole database, with three columns (subject, predicate, object)

Databases and SQL 3/4

- SQL example 1
  - Let's assume we want all actors from the film "Burn After Reading" and that we have a table acted_in, which has two columns film and person

```
SELECT  person
FROM    acted_in
WHERE   film = "Burn After Reading";
```

  - Principle: select those rows from the specified table which satisfy the properties specified in the WHERE clause

Databases and SQL 4/4

- SQL example 2
  - Let's assume that we want all pairs of actors who acted in the same movie and that we have **one big table** triples, which has three columns subject, predicate, and object

```
SELECT  t1.subject, t2.subject, object
FROM    triples AS t1, triples AS t2
WHERE   t1.predicate = "acted_in"
AND     t2.predicate = "acted_in"
AND     t1.object = t2.object
```

  - Principle: selects items from **cross-product** $T_1 \times \cdots \times T_k$ which satisfy the properties specified in the WHERE clause
  - Syntax: use AS for unique names of **copies** of the same table; use t.col to refer to a column col from a table t

SQLite 1/3

- A full-fledged database, easy to install and use
  - On Debian/Ubuntu install with: sudo apt-get install sqlite3
  - Two types of commands … examples on next slides

    SQL commands: must end with a semicolon

    SQLite commands: start with a dot, no semicolon at end
  - Two modes to start SQLite:

    sqlite3 will work on an in-memory database

    sqlite3 <name>.db create database in that file, and if file exists, use database from that file

    Let's read our example tables in SQLite using the commands from the next two slides … it's easy

SQLite 2/3

- Some useful SQLite commands by example
  - Specifies the column separator used for input and output

    .separator " " use Ctrl+V TAB for TAB !
  - Read table from TSV (tab-separated values) file

    .import acted_in.tsv acted_in
  - Execute commands from script file (typical suffix is .sql)

    .read <file with commands>
  - Show execution time of every command

    .timer on

SQLite 3/3

- Some useful SQL commands by example
  - Create a table with a given schema

    ```
    CREATE TABLE acted_in(film TEXT, person TEXT);
    ```

  - Create an index for a column of a table

    ```
    CREATE INDEX acted_in_index_1 ON acted_in(person);
    ```

  - Extract / combine data from tables

    ```
    SELECT * FROM acted_in WHERE … LIMIT 100;
    ```

  - Delete table / index (without error message if it's not there)

    ```
    DROP TABLE IF EXISTS acted_in;
    DROP INDEX IF EXISTS acted_in_index_1;
    ```

SPARQL to SQL Translation 1/5

- Motivation
  - We want to translate a given SPARQL query to a SQL query that gives the desired results on a given database
  - In the following example, we use **one table per predicate**

```
CREATE TABLE acted_in(person TEXT, film TEXT)
CREATE TABLE married_to(person1 TEXT, person2 TEXT)
```

Note: all elements from one table are from one predicate, so we don't need to store the relation name in the table

For ES12, use **one big table** for all the data, with three columns named **subject**, **predicate**, **object**

This is deliberately different from how we did it in the lecture, so that you have to do some thinking yourself

SPARQL to SQL Translation 2/5

- Example
  - SPARQL query

```
SELECT ?person1 ?person2 ?film WHERE {
   ?person1   acted_in   ?film  .
   ?person2   acted_in   ?film  .
   ?person1   married_to   ?person2  .
}
```

  - SQL query:

```
SELECT   a1.person, a2.person, a1.film
FROM     married_to AS m, acted_in AS a1, acted_in AS a2
WHERE    a1.film = a2.film
[AND     a1.person ≠ a2.person]
AND      a1.person = m.person1
AND      a2.person = m.person2
```

not really needed because (usually) a person is not married to themselves

SPARQL to SQL Translation 3/5

- Algorithm
  - It is up to you in ES12, to design a generic algorithm that works for arbitrary basic SPARQL queries or the form

    SELECT <list of variables> WHERE { <list of triples> }

  - The algorithm is not difficult, but requires understanding of how the data is stored and how SPARQL and SQL work in principle

    It is a perfect exercise to understand **the basics** !

    In their full generality, SPARQL and SQL are much more complex, but we omit this for this lecture and ES12

  - On the next slide we give you some valuable advice

SPARQL to SQL Translation 4/5

- Algorithm, advice for ES12
  - If there are k query triples in the SPARQL query, have k entries in the FROM clause of the SQL query

    FROM wikidata as w1, wikidata as w2, ... , wikidata as wk
  - In your code, for each variable from the SPARQL query, build an **array** of all its occurrences in the query, e.g.

    ?x: w1.subject, w2.object, w5.object
  - Then, when building the SQL query, add the corresponding equalities to the WHERE clause, e.g.

    WHERE w1.subject = w2.object AND w2.object = w5.object

    Note: if ?x occurs m times, m – 1 equalities are enough

SPARQL to SQL Translation 5/5

- More advice for ES12 (Python $\leftrightarrow$ SQL)
  - Executing SQL commands to a SQLite database from within **Python** is easy:

```
import sqlite3
db = sqlite3.connect("example.db")
cursor = db.cursor()
cursor.execute("SELECT * FROM table")
for row in cursor.fetchall():
   print("\t".join(row))
```

Beware: the SQLite commands (starting with a dot) cannot be executed from within Python, you need SQLite for those

Performance 1/4

- Cross product of tables

  $|T_i|$ = number of rows in table T_i

  – Recall that, <u>conceptually</u> an SQL statement like

  SELECT … FROM $T_1, T_2, \ldots, T_k$ WHERE …

  selects elements from the **cross-product**

  $T_1 \times \cdots \times T_k$

  where some or all of the T_i can be the same table

  – The number of elements in $T_1 \times \cdots \times T_k$ is: $|T_1| \cdot \ldots \cdot |T_k|$

  An efficient SQL engine does not manifest this product, but uses JOIN operations on the tables … see next slides

Performance 2/4

slight abuse of notation

```
SELECT X.c, X.attr, Y.attr
FROM X, Y
WHERE X.c = Y.c
```

- Joining of tables
  - Each … = … in the WHERE clause effectively asks for a JOIN operation between two tables

    Effectively a list intersection on the JOIN columns
  - If we CREATE an **index** for the respective tables on the respective join columns, this list intersection gets fast

    One way to do this: sort (a copy of) the table by that join column and use our "Zipper" algorithm from Lecture 3

Table X

attr	c
X1	A
X2	B
X3	B
X4	C

pointers from ZIPPER

Table Y

c	attr
A	Y1
C	Y2
C	Y3
D	Y4

JOIN → on column c

Result

c	X.attr	Y.attr
A	X1	Y1
C	X4	Y2
C	X4	Y3

same attr columns

one column

one column

same attr columns

Performance 3/4

- Join ordering
  - Typical SQL-from-SPARQL queries require multiple joins
  - Order of joins can make a **huge** performance difference
  - For example, our acted_in table (person - film) is about ten times larger than our married_to table (person – person)
  - **Join order 1:** look at all pairs of actors who played in the same film, and for each check whether they are married

    materializes all pairs of actors from same film (large)
  - **Join order 2:** look at all married couples and for each get their films and check whether they overlap

    materializes list of films of all married people (small)

Performance 4/4

- Join ordering, continued
  - Without further ado, SQLite seems to take the order of the tables in the FROM clause as its join order

```
SELECT  married_to.person1, married_to.person2
FROM    acted_in AS act_1, acted_in as act_2, married_to
WHERE   married_to.person1 = act_1.person
AND     married_to.person2 = act_2.person
AND     act_1.film = act_2.film;
```

  - The following commands collect statistics regarding the table sizes, which are then used to optimize the join order

```
ANALYZE acted_in;
ANALYZE married_to;
```

References

- Textbook
  - Nothing in the text book by Manning, Raghavan, Schütze
- Wikipedia
  - http://en.wikipedia.org/wiki/Knowledge_base
  - http://en.wikipedia.org/wiki/SPARQL
  - http://en.wikipedia.org/wiki/SQL
  - http://en.wikipedia.org/wiki/SQLite
  - http://en.wikipedia.org/wiki/Freebase
  - https://www.wikidata.org
  - https://www.mediawiki.org/wiki/Wikibase/Indexing/RDF_Dump_Format